

ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ПРОФЕССИОНАЛЬНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ИРКУТСКОЙ ОБЛАСТИ
«АНГАРСКИЙ ТЕХНИКУМ СТРОИТЕЛЬНЫХ ТЕХНОЛОГИЙ»

ДИПЛОМНАЯ РАБОТА

Тема: «Web-приложение для продажи спортивного
инвентаря»

Разработал: _____ Фиузов Д.М.
(Подпись)

Руководитель: _____ Меркулова С.В.
(Подпись)

Ангарск, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	5
1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ВЕБ-ПРИЛОЖЕНИЯ.....	7
1.1. Анализ предметной области и рынка спортивного инвентаря	7
1.1.1. Современное состояние и тенденции рынка спортивных товаров.....	7
1.1.2. Анализ существующих решений (конкурентов)	8
1.1.3. Формулирование требований к разрабатываемому приложению «SportI»	10
1.2. Классификация и архитектура современных веб-приложений.....	11
1.2.1. Определение и эволюция веб-приложений.....	11
1.2.2. Архитектурные подходы.....	12
1.2.3. Модель клиент-сервер и протокол HTTP/HTTPS.....	13
1.3. Обоснование выбора технологического стека для backend-разработки ..	14
1.3.1. Сравнительный анализ серверных платформ	14
1.3.2. Фреймворк Express.js	15
1.3.3. Выбор СУБД: MySQL и её альтернативы	16
1.3.4. Инструменты для работы с СУБД.....	18
1.3.5. Дополнительные библиотеки Node.js	18
1.4. Обоснование выбора технологического стека для frontend-разработки ..	19
1.4.1. Фундаментальные технологии: HTML5, CSS3, JavaScript	19
1.4.2. Выбор шаблонизатора: EJS.....	20

						ДР-09.02.07- Ф-30-26ПЗ										
Изм.	Кол.уч	Лист	№ док.	Подпись	Дата	«Web-приложение для продажи спортивного инвентаря»				<u>Лит.</u>		Лист				
Разраб.		Фнузов Д.М.											3		51	
Пров.		Меркулова С.В.														
Н.контр																
Утв.																
ГАПОУ ИО АТСТ																

1.4.3. Выбор CSS-фреймворка: Bootstrap 5	21
1.5. Принципы проектирования и обеспечения безопасности веб-приложений	22
1.5.1. Основные угрозы безопасности в веб-приложениях	22
1.5.2. Методы и средства защиты	23
1.5.3. Проектирование базы данных и её защита.....	25
2. РАЗРАБОТКА ПРОГРАММНОГО ПРОДУКТА	29
2.1. Подготовка рабочего окружения и создание основы проекта	29
2.2. Настройка сервера Express и шаблонизатора EJS	29
2.4. Реализация регистрации и авторизации пользователей.....	32
2.5. Создание каталога товаров с фильтрацией и пагинацией	33
2.6. Корзина и избранное (без перезагрузки страницы).....	35
2.7. Страница товара и система отзывов.....	38
2.8. Административная панель	40
2.9 Оформление заказа	43
2.10. Личный кабинет	45
2.11. Главная страница	46
ЗАКЛЮЧЕНИЕ	48
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	50
Приложение А	
Приложение Б	

						ДР-09.02.07- Ф-30-26ПЗ			
Изм.	Кол.уч	Лист	№ док.	Подпись	Дата				
Разраб.	Фиузов Д.М.					«Web-приложение для продажи спортивного инвентаря»	Лит.	Лист	
Пров.	Меркулова С.В.							4	51
							ГАПОУ ИО АТСТ		
Н.контр									
Утв.									

ВВЕДЕНИЕ

В условиях стремительного развития цифровых технологий и роста популярности онлайн-торговли рынок спортивного инвентаря претерпевает существенные трансформации. Спортивный образ жизни становится неотъемлемой частью повседневности миллионов людей, что напрямую стимулирует спрос на качественные, современные и разнообразные товары для занятий фитнесом, командными видами спорта, единоборствами и другими направлениями. При этом всё больше потребителей отдают предпочтение онлайн-каналам приобретения товаров - они позволяют не только экономить время, но и сравнивать широкий ассортимент, изучать характеристики, читать отзывы и совершать покупки в удобное время и из любого места с доступом в интернет.

Актуальность разработки веб-приложения для продажи спортивного инвентаря обусловлена следующими факторами:

- устойчивый рост числа людей, ведущих активный образ жизни и регулярно приобретающих спортивные товары;
- изменение потребительских привычек в сторону онлайн-покупок;
- необходимость создания удобной, безопасной и гибкой цифровой платформы, отвечающей современным требованиям электронной коммерции;
- возможность автоматизации бизнес-процессов: управления товарами, заказами, складскими остатками и клиентской базой.

Целью данной дипломной работы является разработка функционального и удобного веб-приложения, ориентированного на продажу спортивного инвентаря. Для достижения поставленной цели определены следующие задачи:

1. Провести анализ существующих решений и технологий, применяемых при создании веб-приложений для электронной коммерции.
2. Сформулировать требования к функциональности, дизайну и архитектуре приложения на основе потребностей целевой аудитории.
3. Разработать прототип пользовательского интерфейса с акцентом на интуитивность и удобство навигации.
4. Создать реляционную базу данных, обеспечивающую эффективное хранение и управление информацией о пользователях, товарах, заказах и категориях.
5. Обеспечить базовую защиту персональных данных и устойчивость приложения к типичным уязвимостям (например, SQL-инъекции, XSS).
6. Реализовать минимально жизнеспособную версию продукта с ключевыми функциями: каталог товаров, корзина, регистрация/авторизация, личный кабинет, административная панель.

В результате выполнения дипломной работы будет создано полноценное веб-приложение, способное стать основой для дальнейшего развития онлайн-магазина спортивного инвентаря.

1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ВЕБ-ПРИЛОЖЕНИЯ

1.1. Анализ предметной области и рынка спортивного инвентаря

1.1.1. Современное состояние и тенденции рынка спортивных товаров

Рынок спортивных товаров является одним из наиболее динамично развивающихся сегментов мировой электронной коммерции. По данным аналитических агентств, в 2024 году глобальный объем рынка спортивных товаров оценивался в 386,53 млрд долларов США, а к 2032 году, по прогнозам, достигнет 736,49 млрд долларов США при среднегодовом темпе роста (CAGR) около 7,8%. Основными драйверами роста выступают:

- Популяризация здорового образа жизни. В последние годы фитнес, бег, йога и другие виды физической активности стали неотъемлемой частью повседневной культуры миллионов людей. Это стимулирует спрос на качественный инвентарь: от гантелей и ковриков до экипировки для командных видов спорта.
- Цифровая трансформация розничной торговли. Потребители всё чаще предпочитают покупать спортивные товары онлайн из-за удобства сравнения цен, широты ассортимента и возможности ознакомиться с отзывами.
- Развитие инфраструктуры доставки и логистики. Быстрая и надежная доставка позволяет интернет-магазинам конкурировать с офлайн-точками.
- Сезонные факторы и проведение крупных спортивных мероприятий. Олимпийские игры, чемпионаты мира по футболу и другие события вызывают всплеск интереса к определённым видам спорта и, как следствие, к сопутствующим товарам.

В России рынок спортивных товаров также демонстрирует устойчивый рост. Согласно исследованию «Infoline», объем российского рынка спортивных товаров в 2023 году составил около 420 млрд рублей, а доля

						ДР-09.02.07-Ф-30-26ПЗ	Лист
							7
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

онлайн-продаж превысила 35%. Наиболее востребованными категориями являются:

- Фитнес-оборудование (беговые дорожки, велотренажеры, гантели, гири).
- Одежда и обувь для спорта и активного отдыха.
- Товары для зимних видов спорта (лыжи, сноуборды, коньки).
- Аксессуары (бутылки для воды, сумки, коврики).

Вместе с тем существуют и проблемы: высокая конкуренция, необходимость обеспечения безопасности платежей и защиты персональных данных, а также сложность с возвратом товаров, которые не подошли по размеру или характеристикам. Именно поэтому разработка качественного, безопасного и удобного веб-приложения для продажи спортивного инвентаря является актуальной и востребованной задачей.

1.1.2. Анализ существующих решений (конкурентов)

Для выявления сильных и слабых сторон существующих интернет-магазинов спортивных товаров проведем сравнительный анализ трёх популярных российских платформ: «Спортмастер», «Триал-спорт» и «Кант». Критерии сравнения выбраны исходя из потребностей целевой аудитории (покупатели спортивного инвентаря) и современных стандартов UX/UI.

Таблица 1. Сравнительный анализ конкурентов

Критерий	«Спортмастер»	«Триал-спорт»	«Кант»
Удобство навигации	Хорошо структурированный каталог, но много рекламных блоков.	Простая, понятная структура, быстрый поиск.	Удобная фильтрация по видам спорта, но не всегда интуитивно.
Функционал корзины и заказа	Стандартный: добавление, удаление, расчёт доставки.	Аналогичен. Есть возможность быстрого заказа.	Расширенный: сравнение товаров, отложенные товары.

Скорость загрузки страниц	Средняя (много тяжелых изображений).	Высокая.	Средняя.
Адаптивность (мобильная версия)	Хорошая, но приложение удобнее.	Отличная.	Удовлетворительная.
Отзывы и рейтинг товаров	Есть, но фильтрация по рейтингу не всегда точна.	Есть, возможность сортировки по рейтингу.	Есть, но отзывов мало.
Наличие личного кабинета и истории заказов	Да.	Да.	Да.
Поиск с фильтрацией	Расширенный (по цене, бренду, размеру).	Расширенный.	Базовый.
Безопасность (HTTPS, защита данных)	Обеспечена.	Обеспечена.	Обеспечена.

Анализ сильных сторон конкурентов:

- «Спортмастер»: широкая сеть пунктов выдачи, программа лояльности.
- «Триал-спорт»: быстрая работа фильтров, удобный поиск.
- «Кант»: ориентация на профессиональный инвентарь, подробные характеристики.

Слабые стороны:

- Медленная работа корзины (требует перезагрузки).
- Отсутствие гибкой админки для управления товарами.
- Много рекламы и навязчивых баннеров.

Вывод: разрабатываемый проект «SportI» ориентирован на скорость, простоту и удобство для администратора. Он будет полезен для небольших спортивных магазинов или стартапов.

1.1.3. Формулирование требований к разрабатываемому приложению «SportI»

На основе анализа предметной области и существующих решений, а также с учётом целей дипломной работы, сформулируем требования к веб-приложению «SportI».

Функциональные требования:

1. Регистрация и авторизация пользователей с хешированием паролей (bcrypt) и разделением ролей («администратор», «пользователь»).
2. Каталог товаров с возможностью просмотра списка товаров, карточки каждого товара, фильтрации по категориям, цене, производителю, а также поиска по названию.
3. Система управления корзиной: добавление товара, изменение количества, удаление, расчёт итоговой суммы.
4. Избранное (список желаний): возможность сохранять понравившиеся товары в отдельный раздел.
5. Личный кабинет пользователя: просмотр профиля, изменение данных, история заказов.
6. Панель администратора:
 - Добавление нового товара (название, описание, цена, категория, изображение, характеристики, скидка).
 - Редактирование и удаление существующих товаров.
 - Добавление новой категории(название, описание, изображение)
7. Система отзывов и оценок для зарегистрированных пользователей. Отображается средний рейтинг товара.
8. Поддержка скидок: возможность устанавливать процент скидки, даты начала и окончания, автоматический расчёт цены со скидкой.

9. Валидация данных на стороне клиента (JavaScript) и сервера (Express-validator или ручная проверка).

Нефункциональные требования:

- Производительность: время отклика страницы не более 2 секунд при стандартной нагрузке.
- Безопасность: защита от XSS, SQL-инъекций (через параметризованные запросы), хеширование паролей, использование HTTPS (в будущем).
- Удобство использования (Usability): интуитивно понятный интерфейс, навигация за 3 клика до любого товара, адаптивная верстка под разные разрешения (Bootstrap 5).
- Масштабируемость: архитектура должна позволять добавлять новый функционал (платежные системы, модуль рассылок и т.д.) без полной переработки кода.
- Поддерживаемость: чистый код с комментариями, использование системы контроля версий Git, файловая структура, разделяющая маршруты, контроллеры, представления и модели.

1.2. Классификация и архитектура современных веб-приложений

1.2.1. Определение и эволюция веб-приложений

Веб-приложение – это программное обеспечение, которое функционирует в среде веб-браузера, использует технологии HTML, CSS, JavaScript и взаимодействует с сервером через сеть Интернет или интранет. В отличие от традиционных десктопных приложений, веб-приложение не требует установки на устройство пользователя, централизованно обновляется и доступно с любой платформы, имеющей браузер.

Эволюция веб-приложений прошла несколько этапов:

						ДР-09.02.07-Ф-30-26ПЗ	Лист 11
Изм.	Кол.уч	Лист	№ докум.	Подпись	Дата		

1. Статические веб-сайты (1990-е гг.) - набор заранее подготовленных HTML-страниц. Контент не изменялся без участия разработчика.
2. Динамические веб-сайты с использованием CGI/Perl/PHP - появление серверной обработки данных, подключение к базам данных (MySQL). Стало возможным создание форумов, гостевых книг, простых интернет-магазинов.
3. Веб-приложения с AJAX (середина 2000-х) - технология асинхронного обмена данными между клиентом и сервером без перезагрузки страницы (Google Maps, Gmail). Резко улучшилось UX.
4. Одностраничные приложения (SPA) - весь интерфейс загружается один раз, дальнейшая навигация происходит через API (React, Angular, Vue). Обеспечивают почти нативное быстроедействие.
5. Прогрессивные веб-приложения (PWA) - могут работать офлайн, отправлять push-уведомления, устанавливаться на рабочий стол, используя сервис-воркеры.

Разрабатываемое приложение «SportI» относится к категории динамических веб-приложений с серверным рендерингом (SSR) на основе шаблонизатора EJS. В перспективе оно может быть дополнено элементами SPA (например, для корзины и избранного через AJAX).

1.2.2. Архитектурные подходы

Современная веб-разработка использует несколько архитектурных парадигм.

- Монолитная архитектура: все компоненты приложения (фронтенд, бизнес-логика, работа с БД) объединены в одном исполняемом модуле. Плюсы: простота разработки и разворачивания, высокая производительность внутренних вызовов.

Минусы: сложность масштабирования отдельных частей, "разрастание" кода. Для небольших и средних проектов (как наш интернет-магазин) монолит – оптимальный выбор.

- Микросервисная архитектура: приложение разбито на множество небольших независимых сервисов, каждый со своей базой данных и API. Плюсы: гибкость масштабирования, независимость команд разработки. Минусы: высокая сложность эксплуатации (оркестрация, сетевое взаимодействие, распределенные транзакции). Для дипломного проекта микросервисы избыточны.
- Serverless архитектура: код выполняется в облачных функциях (AWS Lambda, Google Cloud Functions). Нет необходимости управлять сервером. Подходит для событийно-ориентированных приложений, но может быть дорогой при высокой нагрузке.

Наше приложение «SportI» реализовано как монолитное веб-приложение на базе Node.js + Express с подключением к единой реляционной базе данных MySQL. Такое решение обеспечивает простую разработку, легкое тестирование и развертывание, что полностью соответствует задачам дипломной работы.

1.2.3. Модель клиент-сервер и протокол HTTP/HTTPS

В основе работы любого веб-приложения лежит архитектура клиент-сервер:

- Клиент (обычно веб-браузер) отправляет HTTP-запрос на сервер. Запрос содержит метод (GET, POST, PUT, DELETE), URL, заголовки и, возможно, тело (данные формы, JSON).
- Сервер (физический или виртуальный) принимает запрос, выполняет необходимую логику (обращается к БД, вызывает внешние API), формирует HTTP-ответ (статус, заголовки, тело - HTML, JSON, CSS, изображение) и отправляет его клиенту.

- Протокол HTTP/HTTPS (HyperText Transfer Protocol Secure) - основа передачи данных. HTTPS добавляет шифрование с помощью TLS/SSL, что критически важно для защиты логинов, паролей и платежных данных.

В нашем приложении сервер построен на Node.js с использованием фреймворка Express, который упрощает маршрутизацию и обработку запросов. Взаимодействие с клиентом происходит синхронно (при переходе по ссылкам - полная перезагрузка страницы) и частично асинхронно (через fetch для добавления в избранное без перезагрузки).

1.3. Обоснование выбора технологического стека для backend-разработки

1.3.1. Сравнительный анализ серверных платформ

Для разработки серверной части были рассмотрены следующие популярные платформы:

- Node.js (JavaScript/TypeScript) - событийно-ориентированная, неблокирующая платформа на движке V8.
- Python + Django/Flask - высокоуровневый язык, богатая экосистема для анализа данных и ML.
- PHP + Laravel/Symfony - традиционный выбор для интернет-магазинов (WordPress, Magento).
- C# + ASP.NET Core - мощная корпоративная платформа.

Критерии выбора для нашего проекта: простота освоения, скорость разработки, производительность при обработке большого числа одновременных запросов, возможность использования одного языка (JavaScript) на фронтенде и бэкенде, а также наличие необходимых библиотек.

Таблица 2. Сравнение серверных платформ

Платформа	Производительность (RPS)	Порог входа	JavaScript на фронте и бэке	Экосистема для e-commerce	Подходит для диплома

Node.js	Высокая (неблокирующая I/O)	Средний	Да	Много библиотек (Passport, Sequelize, Express)	Да
Python	Средняя (блокирующая, но с асинхронными фреймворками)	Низкий	Нет	Django- oscar, Saleor	Да, но другой язык усложнит проект
PHP	Средняя (зависит от кэширования)	Низкий	Нет	Laravel + Cashier, много готовых решений	Да, но считается менее современным
ASP.NET Core	Высокая	Высокий	Нет (C#)	Магазины на porCommerce	Слишком сложен для диплома

Обоснование выбора Node.js:

- Высокая производительность при операциях ввода-вывода (чтение файлов, запросы к БД), что критично для интернет-магазина с множеством одновременных пользователей.
- Единый язык (JavaScript) для клиентской и серверной части – уменьшает когнитивную нагрузку, позволяет переиспользовать код (валидаторы, утилиты).
- Огромный репозиторий пакетов npm (более 1,5 млн), включая bcrypt, express-session, formidable, mysql2.
- Простота создания REST API и интеграции с любым фронтендом.

1.3.2. Фреймворк Express.js

Express.js – это минималистичный и гибкий веб-фреймворк для Node.js, который предоставляет набор мощных функций для создания веб-приложений и API. Он является де-факто стандартом для Node.js благодаря своей простоте и расширяемости.

Основные возможности Express.js, используемые в проекте:

- Маршрутизация – определение обработчиков для различных HTTP-методов и URL-путей (app.get, app.post, app.put, app.delete). В проекте маршруты вынесены в отдельный файл index.js.
- Middleware – функции, имеющие доступ к объекту запроса (req) и ответа (res), а также к следующей функции в цикле запрос-ответ. Пример: express.urlencoded() для парсинга данных форм, express.static() для раздачи статики, собственные middleware для проверки аутентификации (isAuth, isAdmin).
- Интеграция с шаблонизаторами – поддержка EJS, Pug, Handlebars. В проекте используется EJS с настройкой app.set('view engine', 'ejs').
- Обработка сессий – через express-session, что позволяет сохранять состояние пользователя между запросами (авторизация, корзина).

Альтернативы Express: Кoa (более легковесный, от создателей Express), Fastify (высокая производительность), NestJS (фулл-стек фреймворк на TypeScript). Для дипломного проекта Express выбран из-за максимальной распространённости, огромного количества обучающих материалов и простоты реализации.

1.3.3. Выбор СУБД: MySQL и её альтернативы

Для хранения данных в веб-приложении необходима система управления базами данных. Рассмотрены два основных типа: реляционные (SQL) и нереляционные (NoSQL).

Реляционные СУБД (MySQL, PostgreSQL, MariaDB, SQLite) – данные организованы в таблицы с фиксированной схемой, связанные внешними ключами. Обеспечивают ACID-транзакции, сложные запросы (JOIN, GROUP BY), целостность данных.

NoSQL СУБД (MongoDB, Redis, Cassandra) – гибкая схема, горизонтальное масштабирование, высокая скорость записи. Но слабая поддержка транзакций и связей между данными.

Поскольку интернет-магазин требует жёсткой согласованности (остатки товаров, заказы, пользователи) и множества связей (товар-категории, товар-отзывы, пользователь-корзина), выбор пал на реляционную СУБД. Из множества решений:

- MySQL – наиболее популярная, отличная документация, простота администрирования, поддержка транзакций (InnoDB), хорошая производительность для чтения. Подходит для учебных и боевых проектов.
- PostgreSQL – более мощная, поддерживает расширенные типы данных (JSONB), но сложнее в настройке. В рамках диплома избыточна.
- SQLite – встраиваемая, файловая, не подходит для многопользовательского приложения с параллельным доступом.

Обоснование выбора MySQL:

- Бесплатность и открытый исходный код.
- Наличие драйвера mysql2 для Node.js с поддержкой промисов и подготовленных выражений.
- Широкая распространённость в хостинговых средах (cPanel, phpMyAdmin) – упрощает будущее развёртывание.
- Поддержка внешних ключей и каскадного удаления (ON DELETE CASCADE).
- Встроенный механизм транзакций (InnoDB), обеспечивающий целостность заказов.

1.3.4. Инструменты для работы с СУБД

Для проектирования, визуализации и администрирования базы данных использовались два инструмента:

- DBeaver Community – универсальный кроссплатформенный клиент с открытым исходным кодом. Поддерживает любую СУБД через JDBC. Позволяет строить ER-диаграммы, редактировать данные, выполнять SQL-скрипты, сравнивать схемы. Использовался на этапе проектирования и отладки запросов.
- phpMyAdmin – веб-интерфейс для управления MySQL, очень популярен на shared-хостинге. В проекте использовался для быстрого просмотра таблиц и выполнения простых запросов при тестировании.

Оба инструмента бесплатны, функциональны и значительно ускоряют разработку.

1.3.5. Дополнительные библиотеки Node.js

В проекте задействован ряд npm-пакетов, расширяющих возможности Node.js:

- bcrypt – библиотека для хэширования паролей. Использует адаптивный алгоритм Blowfish, что усложняет перебор. При регистрации пароль хэшируется с солью (salt rounds = 10), при авторизации введенный пароль сравнивается с хэшем из БД.
- express-session – middleware для управления сессиями. Хранит идентификатор сессии в куке (подписанной и защищенной). В проекте настроено время жизни сессии 24 часа, что позволяет пользователю не входить заново при каждом визите.
- formidable – библиотека для парсинга форм, включая загрузку файлов (изображения товаров, аватары). Позволяет обрабатывать

multipart/form-data. Сохраняет загруженные файлы в папку public/images/products.

- fs (File System) – встроенный модуль Node.js для работы с файловой системой. Используется для удаления старых изображений при обновлении товара, а также для создания директорий.
- mysql2 – драйвер для MySQL с поддержкой промисов и подготовленных выражений. Предотвращает SQL-инъекции через встроенное экранирование.

Кроме того, в проекте используется nodemon для автоматического перезапуска сервера при изменениях в процессе разработки.

1.4. Обоснование выбора технологического стека для frontend-разработки

1.4.1. Фундаментальные технологии: HTML5, CSS3, JavaScript

Любое веб-приложение базируется на трёх столпах:

- HTML5 – язык разметки, отвечающий за структуру документа. Используются семантические теги (<header>, <nav>, <main>, <section>, <article>, <footer>), что улучшает доступность для скринридеров и SEO. В проекте также применяются атрибуты data-* для передачи данных в клиентские скрипты (например, data-product-id).
- CSS3 – каскадные таблицы стилей, описывающие внешний вид. В проекте используется сочетание стандартных CSS-правил и фреймворка Bootstrap 5. Авторские стили (style.css) включают кастомные карточки товаров, горизонтальные карточки для корзины, анимации при наведении. Применяется методология БЭМ (блок-элемент-модификатор) для структурирования имён классов.

- JavaScript (ES6+) – язык программирования, обеспечивающий интерактивность на стороне клиента. В проекте скрипты (script.js) реализуют:
 1. Клиентскую валидацию форм (пароль, email, обязательные поля).
 2. Асинхронную отправку запросов (fetch) для добавления товара в избранное без перезагрузки.
 3. Динамический расчёт рейтинга товара на основе звёзд.
 4. Обработку событий (клики, ввод в поле поиска, отправка формы отзыва).

1.4.2. Выбор шаблонизатора: EJS

Шаблонизатор (view engine) позволяет генерировать HTML на сервере, вставляя динамические данные из переменных. Для Node.js+Express существует несколько популярных решений: EJS, Pug (ранее Jade), Handlebars.

Таблица 3. Сравнение шаблонизаторов

Шаблонизатор	Синтаксис	Плюсы	Минусы
EJS	Внедрение JavaScript в теги <code><% %></code>	Очень похож на обычный HTML, низкий порог входа, полный доступ к JS-логике	Можно намешать много логики в представления
Pug	Отступы, отсутствие закрывающих тегов	Краткий, мощные вложения	Необычный синтаксис, сложнее для фронтендера
Handlebars	Mustache-подобный	Строгое разделение логики и представления (помощники)	Менее гибкий, нужно писать хелперы для каждой операции

Обоснование выбора EJS:

- Разработчик уже знаком с HTML и JavaScript – нет нужды учить новый синтаксис.

- Легко передавать данные из Express в представление (res.render('page', { data })).
- Возможность использовать частичные шаблоны (partials) – например, header.ejs, footer.ejs, product-card.ejs, что упрощает поддержку кода.
- В проекте EJS применяется для рендеринга основного представления layout.ejs и подгрузки в него header.ejs, footer.ejs и текущего «тела» страницы

1.4.3. Выбор CSS-фреймворка: Bootstrap 5

Bootstrap 5 – самый популярный HTML/CSS/JS фреймворк для создания адаптивных, ориентированных на мобильные устройства веб-сайтов. Он предоставляет:

- Сетку на основе flexbox (12 колонок) – позволяет гибко верстать страницы под разные размеры экрана (мобильные, планшеты, десктопы).
- Готовые компоненты – карточки товаров, формы, модальные окна, навигационные панели, пагинация, вкладки.
- Утилиты – отступы, цвета, типографика, выравнивание.
- Иконки Bootstrap Icons (использованы для корзины, избранного, профиля).

Альтернативы: Tailwind CSS (утилитарный, требует больше времени на начальную настройку), Foundation (менее популярен), Materialize (на основе Material Design). Bootstrap выбран из-за простоты использования, наличия русскоязычной документации и совместимости со всеми браузерами. В проекте Bootstrap применён для всех страниц, а кастомные стили дополняют общий дизайн сайта.

1.4.4. Система контроля версий и организация разработки

Git – распределённая система контроля версий, позволяющая отслеживать изменения в исходном коде, работать в команде, возвращаться к предыдущим состояниям. Для работы с Git использовался графический клиент GitHub Desktop, который упрощает основные операции: коммит, пуш, пулл, разрешение конфликтов.

Преимущества использования Git в дипломном проекте:

- История изменений – можно в любой момент посмотреть, что было изменено и когда.
- Безопасность – код хранится на локальном компьютере и на удалённом репозитории (GitHub).
- Ветвление – можно создавать отдельные ветки для экспериментов, не ломая основную версию.
- Демонстрация – ссылка на GitHub-репозиторий может быть включена в пояснительную записку как подтверждение самостоятельной разработки.

Организация файловой структуры в проекте соответствует типовой архитектуре MVC (Model-View-Controller) для Node.js. Такая структура облегчает навигацию по коду и его поддержку.

1.5. Принципы проектирования и обеспечения безопасности веб-приложений

1.5.1. Основные угрозы безопасности в веб-приложениях

При разработке веб-приложения, работающего с персональными данными и деньгами, необходимо учитывать наиболее распространённые уязвимости согласно рейтингу OWASP Top 10:

1. Инъекции (SQL, NoSQL, OS command) – когда злоумышленник передаёт вредоносные данные, которые интерпретируются как часть команды. Особенно опасны SQL-инъекции, позволяющие прочитать, изменить или удалить базу данных.

2. Нарушение аутентификации и управления сессиями – слабые пароли, незащищённые куки, некорректное завершение сессии.
3. Межсайтовый скриптинг (XSS) – внедрение вредоносных JavaScript-скриптов в страницы, которые выполняются в браузере других пользователей.
4. небезопасная прямая ссылка на объект (IDOR) – возможность подобрать идентификатор заказа или пользователя в URL.
5. небезопасная конфигурация – включённый режим отладки, значения по умолчанию, открытые порты.

В нашем проекте особое внимание уделено защите от SQL-инъекций, XSS и правильному управлению сессиями.

1.5.2. Методы и средства защиты

Защита от SQL-инъекций:

- Использование подготовленных выражений (prepared statements) через библиотеку mysql2. Вместо прямого вставления пользовательского ввода в строку запроса, используются плейсхолдеры ? или :name, а значения передаются отдельно. Драйвер автоматически экранирует спецсимволы.

- Пример работы:

```
const [rows] = await pool.query(
  'SELECT * FROM products WHERE productTitle LIKE ?',
  [`%${searchTerm}%`]
);
```

Защита от XSS:

- EJS по умолчанию экранирует данные, переданные в шаблон, с помощью escape – заменяет <, >, &, ", ' на HTML-сущности. Чтобы вывести неэкранированный HTML (например, из

администратора), используется синтаксис `<%- data %>`, но это допускается только для доверенного контента.

- На клиентской стороне при динамическом создании элементов через `innerHTML` также следует использовать `textContent`.

Управление сессиями и аутентификация:

- Хеширование паролей `bcrypt` – даже при утечке базы данных пароли не будут восстановлены.
- Сессия на основе `express-session` с параметрами: `secret` (длинная случайная строка), `resave: false`, `saveUninitialized: false`, `cookie: { httpOnly: true, maxAge: 60000*24 }`. Флаг `httpOnly` запрещает доступ к куке через JavaScript, снижая риск XSS-кражи сессии.
- Для защищённых маршрутов (админка, личный кабинет) реализованы `middleware-функции isAuthenticated` и `isAdmin`, которые проверяют наличие пользователя в сессии и его роль.

Защита от IDOR:

- При обращении к корзине, избранному или заказам всегда проверяется, что текущий авторизованный пользователь имеет право доступа именно к этим данным. Например, корзина загружается по `customerID` из сессии, а не по переданному в URL параметру.

Валидация данных:

- На сервере перед вставкой в БД проверяются все поля: `email` должен быть корректным, цена – числом, текст отзыва – не превышать лимит и не содержать запрещённые теги.
- На клиенте – для удобства пользователя и снижения нагрузки на сервер, но клиентская валидация не должна рассматриваться как основная защита.

1.5.3. Проектирование базы данных и её защита

Проектирование базы данных выполнено в соответствии с принципами нормализации (каждая сущность в своей таблице, минимизация дублирования). Схема включает следующие основные таблицы:

Таблица 4. categories

Название поля	Тип данных	Пример данных
categorieID	INT (AUTO_INCREMENT)	1
categorieName	VARCHAR(100)	«Гантели»
categorieDescription	TEXT	«Компактные снаряды для изолированной проработки мышц...»
categorieThumbnail	VARCHAR(255)	«Гантеля .png»

Таблица 5. categoriesandproducts

Название поля	Тип данных	Пример данных
categoriesandproductsID	INT (AUTO_INCREMENT)	36
categorieID	INT (внешний ключ → categories.categorieID)	1
productID	INT (внешний ключ → products.productID)	2

Таблица 6. customers

Название поля	Тип данных	Пример данных
customerID	INT (AUTO_INCREMENT)	2
customerName	VARCHAR(50)	«User»
customerEmail	VARCHAR(45)	«admin@mail.ru»
customerPassword	VARCHAR(100)	«2b2b10\$MxZIE6Ut...»
customerThumbnail	VARCHAR(255)	«/images /image.jpg»
customerRank	VARCHAR(20)	«admin», «user»

Таблица 7. delivery_points

Название поля	Тип данных	Пример данных
pointID	INT (AUTO_INCREMENT)	1
pointName	VARCHAR(100)	«Пункт выдачи №1»
address	VARCHAR(255)	«г. Москва, ул. Тверская, д. 1»
latitude	DECIMAL(10,8)	55.75580000
longitude	DECIMAL(11,8)	37.61730000

work_hours	VARCHAR(100)	«Пн-Вс: 9:00-21:00»
phone	VARCHAR(20)	«+7 (495) 123-45-67»
is_active	TINYINT(1)	1

Таблица 8. favorites

Название поля	Тип данных	Пример данных
favoritesID	INT (AUTO_INCREMENT)	1
customerID	INT (внешний ключ → customers.customerID)	2
productID	INT (внешний ключ → products.productID)	7

Таблица 9. order_status

Название поля	Тип данных	Пример данных
statusID	INT (AUTO_INCREMENT)	1
statusName	VARCHAR(50)	«В обработке»
statusDescription	VARCHAR(255)	«Заказ принят и ожидает обработки»

Таблица 10. orders

Название поля	Тип данных	Пример данных
orderID	INT (AUTO_INCREMENT)	1
customerID	INT (внешний ключ → customers.customerID)	2
totalAmount	DECIMAL(10,2)	1359.20
deliveryType	ENUM('pickup','delivery')	'pickup'
deliveryAddress	VARCHAR(255)	«ул Пушкина д.2»
deliveryPointID	INT (внешний ключ → delivery_points.pointID)	1
deliveryCost	DECIMAL(10,2)	0.00
paymentType	ENUM('cash','card','online')	'cash'
paymentStatus	ENUM('pending','paid','cancelled')	'pending'
orderStatusID	INT (внешний ключ → order_status.statusID)	1
customerName	VARCHAR(100)	«User»
customerPhone	VARCHAR(20)	«+7 (950) 129-15-76»
customerEmail	VARCHAR(100)	«dani1228lol12@mail.ru»
comment	TEXT	«комментарий»
qrCodeData	TEXT	NULL
paymentLink	VARCHAR(255)	NULL
orderDate	TIMESTAMP	«2026-02-14 06:13:33»

Таблица 11. order_items

Название поля	Тип данных	Пример данных
---------------	------------	---------------

orderItemID	INT (AUTO_INCREMENT)	1
orderID	INT (внешний ключ → orders.orderID)	1
productID	INT (внешний ключ → products.productID)	7
quantity	INT	1
price	DECIMAL(10,2)	1699.00
discountedPrice	DECIMAL(10,2)	1359.20
created_at	TIMESTAMP	«2026-02-14 06:13:33»

Таблица 12. products

Название поля	Тип данных	Пример данных
productID	INT (AUTO_INCREMENT)	2
productTitle	VARCHAR(100)	«Гантель Demix, 2 кг»
productDescription	VARCHAR(500)	«Гантель эргономичной формы со скругленными краями...»
productThumbnail	VARCHAR(100)	«Demix2kg.jpg»
productManufacturer	VARCHAR(100)	«Demix»
productPrice	DECIMAL(10,2)	899.00
discount_percentage	DECIMAL(5,2)	10.00
discount_price	DECIMAL(10,2)	899.10
discount_start_date	DATETIME	«2025-10-11 01:54:37»
discount_end_date	DATETIME	«2025-10-12 01:54:37»
is_on_sale	TINYINT(1)	0
productRating	DOUBLE	0
stock_quantity	INT	15
is_available	TINYINT	1
created_at	TIMESTAMP	«2025-12-11 01:54:37»

Таблица 13. product_features

Название поля	Тип данных	Пример данных
featureID	INT (AUTO_INCREMENT)	5
productID	INT (внешний ключ → products.productID)	3
feature_key	VARCHAR(100)	«Вес»
feature_value	VARCHAR(500)	«0.5кг»
created_at	TIMESTAMP	«2025-12-11 01:55:57»

Таблица 14. product_images

Название поля	Тип данных	Пример данных
imageID	INT (AUTO_INCREMENT)	1
productID	INT (внешний ключ → products.productID)	2

2. РАЗРАБОТКА ПРОГРАММНОГО ПРОДУКТА

2.1. Подготовка рабочего окружения и создание основы проекта

Разработка веб-приложения начинается с установки необходимого программного обеспечения. На рабочей станции были установлены:

- Node.js (версия 22 и выше) - среда выполнения JavaScript на сервере;
- MySQL Server (версия 8.0) - реляционная база данных;
- DBeaver Community - для визуального управления БД;
- Visual Studio Code - редактор кода с поддержкой отладки.

Далее был создан каталог проекта и инициализирован Node.js-проект командой `npm init -y`.

Установлены следующие зависимости:

```
npm install express ejs mysql2 bcrypt express-session formidable
npm install --save-dev nodemon
```

Рис. 2 Установка зависимостей

- express - веб-фреймворк;
- ejs - шаблонизатор;
- mysql2 - драйвер для MySQL с поддержкой промисов;
- bcrypt - хеширование паролей;
- express-session - управление сессиями;
- formidable - обработка загрузки файлов и данных форм;
- nodemon - автоматический перезапуск сервера при изменениях.

2.2. Настройка сервера Express и шаблонизатора EJS

В файле `app.js` была создана базовая конфигурация сервера. Подключены необходимые модули, настроен `express.urlencoded()` для парсинга данных форм и `express.json()` для обработки AJAX-запросов. Статические файлы из папки `public` сделаны доступными для браузера.

```
const express = require('express');
const session = require('express-session');
const path = require('path');

const app = express();

app.set('view engine', 'ejs');
app.set('views', [path.join(__dirname, 'views')]);

app.use(express.static('public'));
app.use(express.urlencoded({ extended: true }));
app.use(express.json());
```

Рис. 3 Импорт модулей и настройка статике

Шаблонизатор EJS был настроен для использования layout-подхода. Создан файл layouts.ejs, который содержит общую структуру страницы: <head> с подключением Bootstrap 5, кастомного CSS, заголовков, навигационную панель (header), динамическую область для контента (<%-body %>) и подвал (footer). Все остальные представления наследуют этот layout.

```
<div id="animatedBackground">

  <!-- Header -->
  <%- include('header'); %>
  <!-- Alert -->
  <%- include('partials/flash'); %>
  <!-- Body -->
  <%- include(body); %>
  <!-- Footer -->
  <%- include('footer'); %>

</div>
```

Рис. 4 Содержимое layout.ejs

2.3. Проектирование и создание базы данных MySQL

На основе анализа предметной области была спроектирована реляционная база данных. ER-диаграмма включает следующие таблицы:

- customers - пользователи (логин, хеш пароля, роль, аватар);
- products - товары (название, описание, цена, скидка, рейтинг, остаток);
- categories - категории;

- categoriesandproducts - связь товаров и категорий (многие ко многим);
- shopping_cart - корзина (пользователь → товар, количество);
- favorites - избранное;
- reviews - отзывы (рейтинг, текст, дата);
- product_features - характеристики товара (ключ-значение);
- orders и order_items - заказы и состав заказов;
- delivery_points - пункты самовывоза;
- order_status - справочник статусов заказов.

Для создания этих таблиц вводим SQL скрипты в терминал, для примера рассмотрим таблицу categories:

```
CREATE TABLE `categories` (
  `categorieName` varchar(100) COLLATE utf8mb4_general_ci NOT NULL,
  `categorieDescription` text COLLATE utf8mb4_general_ci,
  `categorieThumbnail` varchar(255) COLLATE utf8mb4_general_ci DEFAULT NULL,
  `categorieID` int NOT NULL AUTO_INCREMENT,
  PRIMARY KEY (`categorieID`)
) ENGINE=InnoDB AUTO_INCREMENT=8 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

Рис. 7 SQL скрипт для создания таблицы categories

	A-z categorieName	A-z categorieDescription	A-z categorie	123 categorieID
1	Гантели	Компактные снаряды для изолиров	cat_17783366468	1
2	Гири	Чугунные снаряды с рукоятью для	cat_17783366572	2
3	Блины	Сменные диски для штанги, позвс	cat_17783365983	3
4	Грифы	Прочные металлические стержни,	cat_17783440286	6
5	Утяжелители	Манжеты, пояса и жилеты для пов	cat_17783976631	7

Рис. 6 Готовая таблица categories

Для подключения к БД был создан файл database.js. В нём настроено соединение через mysql2.createConnection()

```
// Подключение к БД
const connection = mysql.createConnection({
  host: process.env.MYSQLHOST,
  port: process.env.MYSQLPORT,
  user: process.env.MYSQLUSER,
  password: process.env.MYSQLPASSWORD,
  database: process.env.MYSQLDATABASE
});
```

Рис. 7 Подключение к базе данных MySQL

Далее были созданы функции обращений к базе данных, вот пример одной из функций:

```
// ===== Главная страница =====
function getAllCategories(callback) {
    connection.query('SELECT * FROM categories ORDER BY categorieName', callback);
}
```

Рис. 8. Функция getAllCategories()

2.4. Реализация регистрации и авторизации пользователей

Для аутентификации использовались сессии (express-session) и хеширование паролей (bcrypt). Маршруты для отображения форм регистрации и входа были добавлены в routes/index.js:

```
router.get('/login', (req, res) => res.render('layout', { body: 'login' }));
router.get('/register', (req, res) => res.render('layout', { body: 'register' }));
```

Рис. 9. Get маршруты аутентификации

При POST-запросе на /register данные из формы валидируются, пароль хешируется, после чего вызывается функция createUser из database.js. Аналогично при /login проверяется существование пользователя и сравнивается пароль.

```
const hashedPassword = bcrypt.hashSync(password, 10);
connection.findUserByUsername(email, (err, results) => {
    if (results?.length) return res.render('layout', { error: 'Такой пользователь уже есть!', body: 'register' });
    connection.createUser(username, hashedPassword, email, (err) => {
        if (err) return res.render('layout', { error: 'Ошибка регистрации!', body: 'register' });
        res.redirect('/');
    });
});
```

Рис. 10 Функция добавления нового пользователя

На стороне клиента для общения с сервером, авторизацией и регистрацией, созданы представления register.ejs и login.ejs

Регистрация

Имя
Имя

Email
Введите email

Пароль
Придумайте пароль

Подтверждение пароля
Повторите пароль

Регистрация

Уже есть аккаунт? [Войти](#)

Рис. 11 Форма регистрации

Для защиты приватных страниц, доступ к которым должен быть только у авторизованных пользователей (корзина, избранное, админка), была написана middleware isAuthenticated:

```
const isAuthenticated = (req, res, next) => {
  if (!req.session.userId) return res.redirect('/login');
  next();
};
```

Рис. 12 middleware isAuthenticated

2.5. Создание каталога товаров с фильтрацией и пагинацией

Маршрут /products обрабатывает GET-запросы, извлекает из строки запроса параметры фильтров (минимальная и максимальная цена, выбранные категории, сортировка, страница). Функция getCatalogProducts из database.js формирует динамический SQL-запрос с учётом всех условий и возвращает товары с постраничным выводом (limit/offset).

```
function getCatalogProducts(customerID, filters, callback) {
  getProductsFiltered(filters, (err, products) => {
    if (err) return callback(err);
    if (!customerID) return callback(null, { products, favorites: [], cart: {}, cartCount: 0 });
    getFavoritesByCustomerID(customerID, (err, favorites) => {
      if (err) favorites = [];
      getCartByCustomerID(customerID, (err, cartItems) => {
        if (err) cartItems = [];
        const cart = {};
        cartItems.forEach(item => { cart[item.productID] = { sc_count: item.sc_count }; });
        const cartCount = Object.values(cart).reduce((sum, i) => sum + (i.sc_count || 0), 0);
        callback(null, { products, favorites: favorites.map(f => f.productID), cart, cartCount });
      });
    });
  });
}
```

Рис. 13 Функция getCatalogProducts

На стороне клиента фильтры реализованы в виде боковой панели. Пользователь может выбрать категории, задать диапазон цены (слайдер), включить чекбоксы «Только со скидкой» и «В наличии». При изменении любого фильтра форма отправляется методом GET, и страница перезагружается с новыми параметрами.

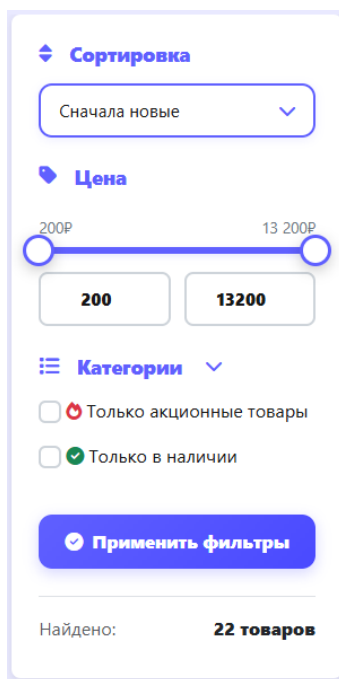


Рис. 14 Боковая панель сортировки

Так же для удобства пользователей был реализован поиск с подсказками при вводе текста. На стороне сервера создан эндпоинт `/api/search-suggestions`, который принимает параметр `query` и возвращает JSON-массив товаров (до 8 записей) с названием, ценой, миниатюрой и рейтингом.

```
router.get('/api/search-suggestions', (req, res) => {
  const query = req.query.query || '';
  const isPopular = req.query.popular === 'true';
  if (isPopular && !query) {
    connection.getPopularSearchSuggestions(5, (err, results) => {
      res.json(err ? [] : formatSuggestions(results));
    });
    return;
  }
  if (query.length < 2) return res.json([]);
  connection.searchProducts(query, 8, (err, results) => {
    res.json(err ? [] : formatSuggestions(results));
  });
});
```

Рис. 15 Маршрут `/api/search-suggestions`

На клиенте в файле script.js написана функция `initSearchForm()`, которая добавляет обработчики ввода текста, отправляет `fetch`-запрос при задержке 300 мс и отображает выпадающий список подсказок. Каждый элемент подсказки содержит ссылку на страницу товара.

```
function fetchSuggestions(query) {
  fetch(`/api/search-suggestions?query=${encodeURIComponent(query)}`)
    .then(response => response.json())
    .then(suggestions => {
      displaySuggestions(suggestions);
    })
    .catch(error => {
      console.error('Error fetching suggestions:', error);
      hideSuggestions();
    });
}
```

Рис. 16 Функция `fetchSuggestions`

На стороне клиента получился поиск с выпадающим списком подсказок



Рис. 17 Поиск с подсказками

2.6. Корзина и избранное (без перезагрузки страницы)

Корзина и избранное реализованы с использованием AJAX-запросов, чтобы не перезагружать страницу при добавлении товара. В `database.js` созданы функции `addToCart`, `removeFromCart`, `updateCartItem`, а для избранного - `addToFavorites` и `removeFromFavorites`.

```
// ===== Корзина =====
function addToCart(customerID, productID, callback) {
  connection.query('INSERT INTO shopping_cart (customerID, productID, sc_count)
    VALUES (?, ?, 1) ON DUPLICATE KEY UPDATE sc_count = sc_count + 1', [customerID, productID], callback);
}
```

Рис. 18 Функция addToCart

На клиенте в script.js определены глобальные функции addToCart(productID) и toggleFavorite(event, productID). Они отправляют POST-запросы на соответствующие маршруты (/cart/add, /favorites/add и т.д.). При успешном ответе обновляется иконка (сердечко или корзина) и счётчик товаров.

```
function addToCart(productID) {
  fetch('/cart/add', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({ productID: productID })
  })
  .then(response => {
    if (response.status === 401) {
      window.location.href = '/login';
      return Promise.reject('Not authorized');
    }
    return response.json();
  })
  .then(data => {
    if (data.success) {
      const cartBadge = document.querySelector('.nav-badge');
      if (cartBadge) {
        cartBadge.textContent = data.cartCount || 0;
      }
      showNotification('Товар добавлен в корзину', 'success');
    } else {
      showNotification(data.message || 'Не удалось добавить товар в корзину', 'error');
    }
  })
  .catch(error => {
    console.error('Cart error:', error);
    if (error !== 'Not authorized') {
      showNotification('Произошла ошибка при добавлении товара в корзину', 'error');
    }
  });
}
```

Рис. 19 Функция addToCart() с fetch на клиентской части

Счётчик товаров в корзине отображается в виде бейджа рядом с иконкой корзины в header. Данные о количестве передаются через res.locals.userCartCount в layout и обновляются после каждого AJAX-запроса.

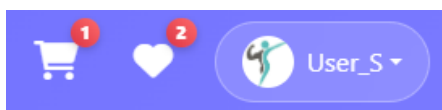


Рис. 20 Бейджи количества избранного и товаров в корзине

Страница корзины состоит из блока «товары в корзине» и «итог заказа», так же реализованы кнопки управления товарами в корзине.

```
<!-- Список товаров -->
<div class="col-lg-8">
  <div class="card cart-products-card no-hover" style="min-height: 50vh;">
    <div class="cart-products-header">
      <h3>
        <i class="fas fa-boxes"></i>Товары в корзине
      </h3>
    </div>
    <div class="card-body p-0">
      <div class="list-group list-group-flush">
        <% let totalPrice = 0; %>
        <% let originalTotal = 0; %>
        <% products.forEach(product => { %>
          <%
            // Вычисление цен
            const productPriceFull = parseFloat(product.productPrice) || 0;
            const discountedPrice = parseFloat(product.discountedPrice) || 0;
            const itemPrice = product.isDiscounted && discountedPrice > 0 ? discountedPrice : productPriceFull;
            const itemTotal = itemPrice * product.sc_count;
            const itemOriginal = productPriceFull * product.sc_count;

            // Состояния наличия
            const isOutOfStock = product.stock_quantity <= 0;
            const isLowStock = product.sc_count > product.stock_quantity;
            const canIncrease = product.sc_count < product.stock_quantity;

            <%
              totalPrice += itemTotal; %>
              originalTotal += itemOriginal; %>
            <div class="cart-item <%= isOutOfStock ? 'out-of-stock' : '' %>">...
            </div>
            <% }) %>
          <% const discountAmount = originalTotal - totalPrice; %> <!-- считаем после цикла -->
        </div>
      </div>
    </div>
  </div>
```

Рис. 21 EJS код блока «товары в корзине»

На клиенте получился такой результат

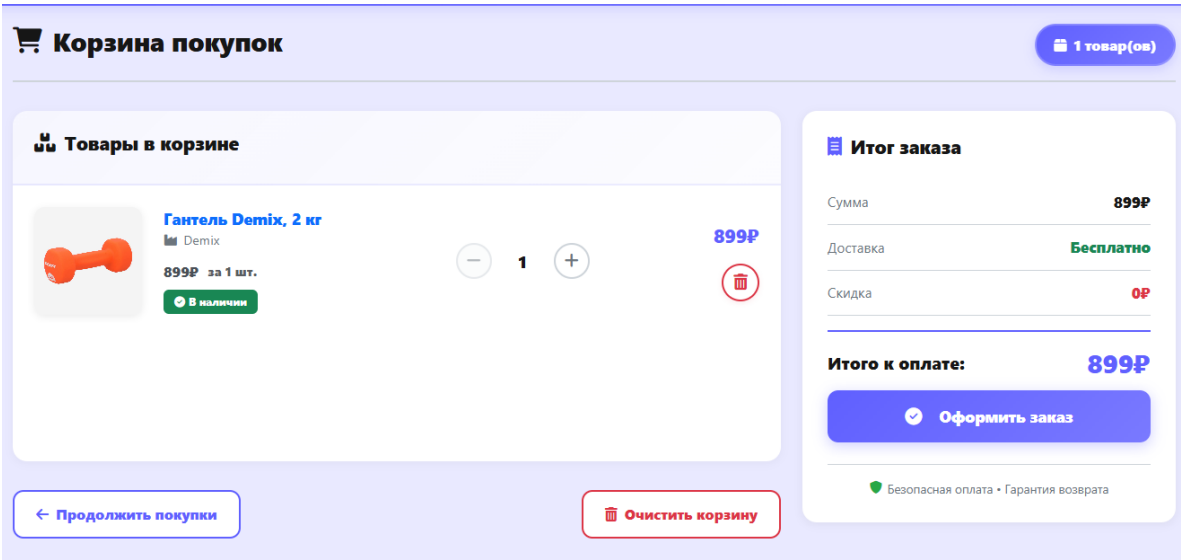


Рис. 22 Страница корзины

2.7. Страница товара и система отзывов

Маршрут /product/:id динамически загружает данные о товаре, его характеристики, категории и отзывы. Страница товара состоит из главного блока с изображением и блоком с основной информацией о товаре(название, цена, категория и кнопки взаимодействия):

```
router.get('/product/:id', async (req, res) => {
  const productId = req.params.id;
  connection.getProductByID(productId, (err, product) => {
    if (err || !product) return res.status(404).send('Товар не найден');
    const priceInfo = connection.calculateDiscountedPrice(product);
    const categoryQuery = `SELECT c.categorieName, c.categorieThumbnail FROM categories c
    JOIN categoriesandproducts cp ON c.categorieID = cp.categorieID WHERE cp.productID = ?`;
    connection.connection.query(categoryQuery, [productId], (catErr, categories) => {
      connection.getProductFeatures(productId, (featErr, features) => {
        connection.getProductReviews(productId, (revErr, reviews) => {
          connection.getReviewStats(productId, (statsErr, stats) => {
            const statsMap = { 5:0,4:0,3:0,2:0,1:0 };
            stats?.forEach(s => { if (s.rating >= 1 && s.rating <= 5) statsMap[s.rating] = { count: s.count, percent
            const overallRating = product.productRating || (reviews.length ? reviews.reduce((s, r) => s + r.rating,
            if (req.session.userId) { ...
          } else { ...
        });
      });
    });
  });
});
```

Рис. 23 Маршрут /product/:id

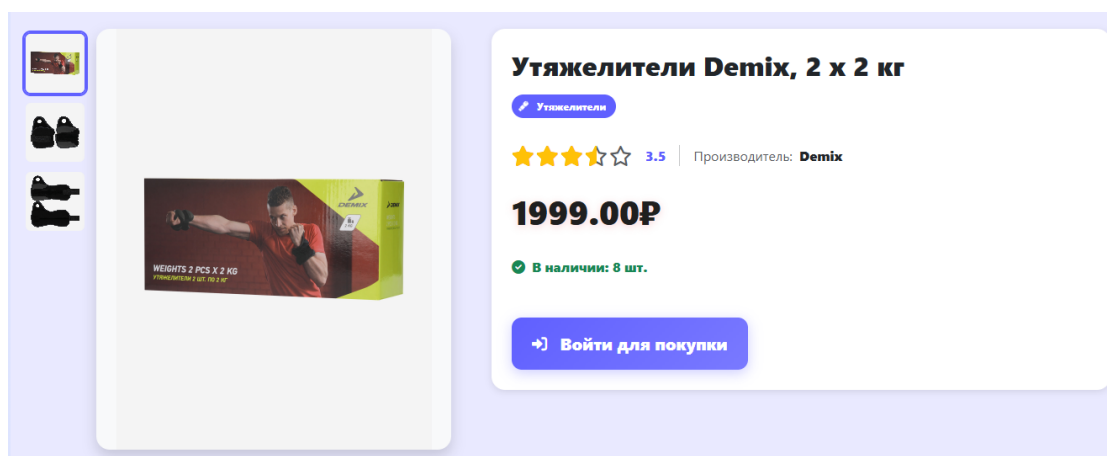


Рис. 24 Верхняя часть страница товара

Ниже на странице товара расположен блок с описанием товара и его характеристиками:

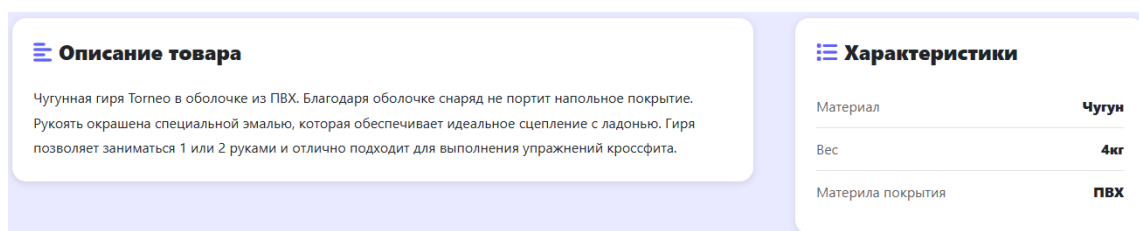


Рис. 25 Описание и характеристики товара

На странице реализована форма отправки отзыва (только для авторизованных пользователей). Пользователь выбирает количество звёзд (1–5) и пишет комментарий. Отправка формы происходит через fetch, после успеха список отзывов обновляется без перезагрузки страницы. Также отображается распределение оценок.

```
<!-- Отзывы -->
<div id="reviews-section" class="mt-5">
  <div class="d-flex justify-content-between align-items-center mb-4">
    <h2 class="h3 fw-bold">
      <i class="fas fa-comments text-primary me-2"></i>Отзывы покупателей
    </h2>
    <% if (reviews.length > 0) { %>
      <span class="badge bg-primary bg-opacity-10 text-primary border border-primary border-opacity-25 px-3 py-2">
        <%= reviews.length %> отзывов
      </span>
    <% } %>
  </div>

  <div class="row g-4">
    <!-- Статистика -->
    <div class="col-lg-4">...
  </div>

    <!-- Список отзывов -->
    <div class="col-lg-8">...
  </div>
</div>

<!-- Модальное окно для отзыва -->
<div class="modal fade" id="reviewModal" tabindex="-1" aria-hidden="true">...
```

Рис. 26 EJS код блока с отзывами

В результате получился такой блок с отзывами, где пользователь может посмотреть из каких отзывов формируется рейтинг товара и если он авторизован, то может оставить отзыв

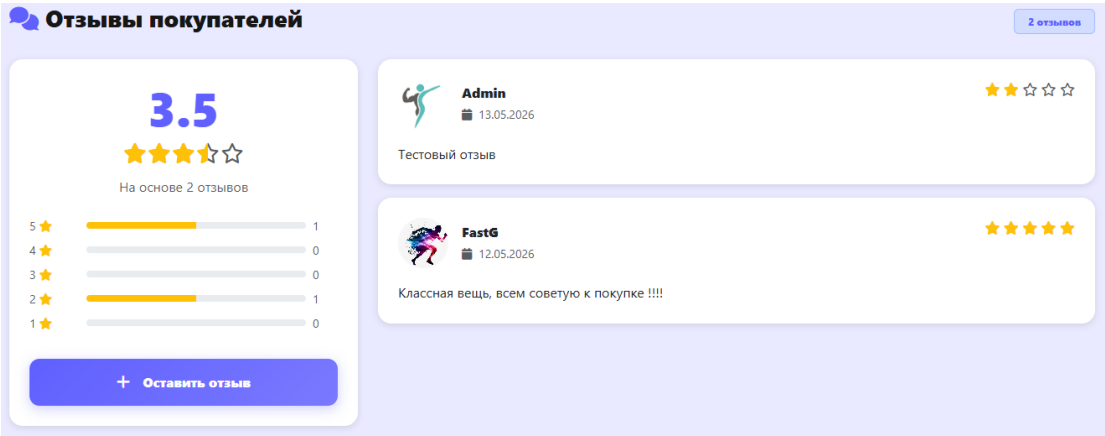


Рис 27. Блок с отзывами

2.8. Административная панель

Для разграничения прав в таблице customers добавлено поле customerRank со значениями user или admin. При входе пользователя в сессию сохраняется его роль. Создана middleware checkAdmin, которая проверяет req.session.userRank.

```
const checkAdmin = (req, res, next) => {
  if (!req.session.userId) return res.redirect('/login');
  connection.getUserRank(req.session.userId, (err, results) => {
    if (err || !results.length || results[0].customerRank !== 'admin') {
      return res.status(403).render('layout', { body: 'error', error: 'Доступ запрещён' });
    }
    next();
  });
};
```

Рис. 28 middleware checkAdmin

Администратору необходимы следующие страницы:

- admin_products - список всех товаров и категорий с поиском, пагинацией и кнопками «Редактировать» / «Удалить».
- add_product - форма добавления нового товара (название, описание, цена, загрузка изображения, выбор категорий, скидка, характеристики).
- edit_product - форма изменения существующего товара (название, описание, цена, загрузка изображения, скидка, выбор категорий, характеристики).
- add_category - добавление новой категории(название, иконка)
- edit_category - изменение существующей категории(название, иконка)

Главная страница администратора располагается по маршруту /admin/products

```
router.get('/admin/products', checkAdmin, (req, res) => {
  const searchQuery = req.query.search || '';
  const page = parseInt(req.query.page) || 1;
  const limit = parseInt(req.query.limit) || 10;
  const offset = (page - 1) * limit;
  connection.connection.query('SELECT COUNT(*) as total FROM products', (err, countResult) => {
  });
});
```

Рис. 29 middleware checkAdmin

Товары 22						+ Добавить товар	
Поиск по названию, производителю или описанию...						Найти	
ID	Товар	Категории	Цена	Рейтинг	Хар-ки		
#2	 Гантель Demix, 2 кг Demix	Гантели	899.00Р	0.0 ★	2		
#3	 Гантель Demix, 0.5 кг Demix	Гантели	299.00Р	4.0 ★	2		
#4	 Гантель Demix, 5 кг Demix	Гантели	1899.00Р	0.0 ★	2		
#5	 Гантель Demix, 3 кг Demix	Гантели	1499.00Р	0.0 ★	2		
#6	 Гантель Demix, 1 кг Demix	Гантели	499.00Р	0.0 ★	2		

Рис. 30 Страница admin_products

Созданы страницы добавления и изменения товара на которых создан функционал изменения названия, цены, скидки, характеристик, описания и изображения товара.

```

<form id="addProductForm" enctype="multipart/form-data" novalidate>
  <h5 class="text-primary mb-3 pb-2 border-bottom"><i class="fas fa-info-circle me-2"></i>Основная информация</h5>
  <div class="row"> ...
  </div>

  <h5 class="text-primary mb-3 pb-2 border-bottom"><i class="fas fa-percentage me-2"></i>Скидка и акции</h5>
  <div class="row"> ...
  </div>
  <div id="discountFields" style="display: none;"> ...
  </div>

  <h5 class="text-primary mb-3 pb-2 border-bottom"><i class="fas fa-image me-2"></i>Изображения товара</h5>
  <div class="mb-3"> ...
  </div>

  <h5 class="text-primary mb-3 pb-2 border-bottom"><i class="fas fa-tags me-2"></i>Категории товара</h5>
  <div class="mb-3"> ...
  </div>

  <h5 class="text-primary mb-3 pb-2 border-bottom d-flex justify-content-between align-items-center"> ...
  </h5>
  <div id="featuresContainer">
    <div class="feature-item mb-3 border rounded p-3 bg-light"> ...
    </div>
  </div>

```

Рис. 31 EJS Форма добавления товара

Загрузка изображений реализована с помощью библиотеки formidable. При добавлении товара файл сохраняется в public/images/products/, так же можно загрузить несколько изображений и выбрать какое будет главным. url главного изображения заносится в productThumbnail таблицы products, а url остальных изображений помещается в таблицу product_images и связываются с товаром через первичный ключ.

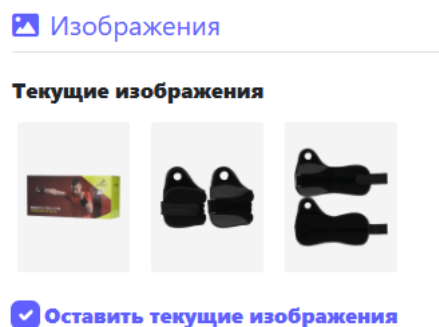


Рис. 32 Изменение изображения

Страницы добавления и изменения категорий обладают меньшим функционалом – добавить название, описание и иконку категории.

```
<div class="card-body p-4">
  <form id="categoryForm" enctype="multipart/form-data">
    <div class="mb-3">
      <label class="form-label fw-semibold">Название категории *</label>
      <input type="text" class="form-control" name="categorieName" required value="%= category.categorieName %">
    </div>
    <div class="mb-3">
      <label class="form-label fw-semibold">Описание</label>
      <textarea class="form-control" name="categorieDescription" rows="3">%= category.categorieDescription || '' %</textarea>
    </div>
    <div class="mb-3"> ...
    </div>
    <div class="mb-3" id="newImageField" style="display: none;"> ...
    </div>
    <div class="d-flex justify-content-between pt-3 border-top">
      <a href="/admin/products" class="btn btn-secondary"><i class="fas fa-arrow-left me-2"></i>Назад</a>
      <button type="submit" class="btn btn-custom px-4" id="submitBtn"><i class="fas fa-save me-2"></i>Сохранить</button>
    </div>
  </form>
  <div id="notificationContainer" class="mt-3"></div>
</div>
```

Рис. 33 EJS Форма изменения категории

На клиенте это выглядит так:

Рис. 34 Форма добавления категории на сайте

2.9 Оформление заказа

Процесс оформления заказа начинается со страницы корзины. Пользователь нажимает кнопку «Оформить заказ» и переходит на /checkout. На этой странице отображается список товаров, общая сумма и форма для ввода контактных данных (ФИО, телефон, email), выбора типа доставки (пункт выдачи или курьерская доставка) и способа оплаты (наличные / карта при получении).

```
<!-- Форма оформления заказа -->
<form id="checkoutForm" class="bg-white rounded-3 shadow-sm p-4 mb-4">
  <!-- Контактная информация -->
  <div class="mb-4"> ...
  </div>

  <!-- Способ получения -->
  <div class="mb-4"> ...
  </div>

  <!-- Способ оплаты -->
  <div class="mb-4"> ...
  </div>

  <!-- Комментарий -->
  <div class="mb-4"> ...
  </div>
</form>
```

Рис. 35 EJS форма оформления заказа

Оформление заказа

Пожалуйста, заполните все необходимые поля для оформления заказа

Контактная информация

ФИО * User_S
Укажите ваше полное имя

Телефон * +7 () - - -
Для связи с вами

Email * daniil228lol12@mail.ru
Для отправки информации о заказе

Способ получения

☒ **Самовывоз**
Бесплатно, из пункта выдачи

☐ **Доставка**
Бесплатно, курьером

Выберите пункт выдачи

☒ **Пункт выдачи №1**

☐ **Пункт выдачи №2**

Итог заказа

Гантель Demix, 2 кг 1 шт. x 899.00₽	899.00₽
Сумма	899.00₽
Доставка	Бесплатно
Скидка	0₽
Итого к оплате:	899.00₽

Безопасная оплата • Гарантия возврата

Рис. 36 Страница оформления заказа на клиенте

После отправки формы данные передаются на /checkout/process. Сервер проверяет наличие товаров на складе (функция checkProductAvailability), затем в одной транзакции создаёт записи в таблицах orders и order_items, а также очищает корзину пользователя.

```
router.post('/checkout/process', isAuthenticated, (req, res) => {
  const { deliveryType, deliveryPointID, deliveryAddress, paymentType, customerName, customerPhone, customerEmail, comment } = req.body;
  if (!deliveryType || !paymentType || !customerName || !customerPhone || !customerEmail) {
    return res.status(400).json({ success: false, message: 'Заполните все обязательные поля' });
  }
  if (deliveryType === 'pickup' && !deliveryPointID) return res.status(400).json({ success: false, message: 'Выберите пункт выдачи' });
  if (deliveryType === 'delivery' && (!deliveryAddress || deliveryAddress.trim() === '')) return res.status(400).json({ success: false, message: 'Введите адрес доставки' });
  connection.getCartWithProducts(req.session.userId, (err, products) => {
    if (err || products.length === 0) return res.status(400).json({ success: false, message: 'Корзина пуста' });
    for (const product of products) {
      if (product.stock_quantity <= 0) return res.status(400).json({ success: false, message: `Товар "${product.productTitle}" отсутствует` });
      if (product.sc_count > product.stock_quantity) return res.status(400).json({ success: false, message: `Недостаточно "${product.productTitle}"` });
    }
    let totalAmount = 0;
    const orderItems = products.map(product => {
      const itemPrice = product.isDiscounted ? parseFloat(product.discountedPrice) : parseFloat(product.productPrice);
      totalAmount += itemPrice * product.sc_count;
      return { productID: product.productID, quantity: product.sc_count, price: parseFloat(product.productPrice), discountedPrice: product.discountedPrice };
    });
    // ... далее обработка заказа
  });
});
```

Рис. 37 Проверка сервера на наличие товара.

Если сервер в результате проверки дает положительный результат тогда пользователя перенаправляет на страницу успешного оформления заказа

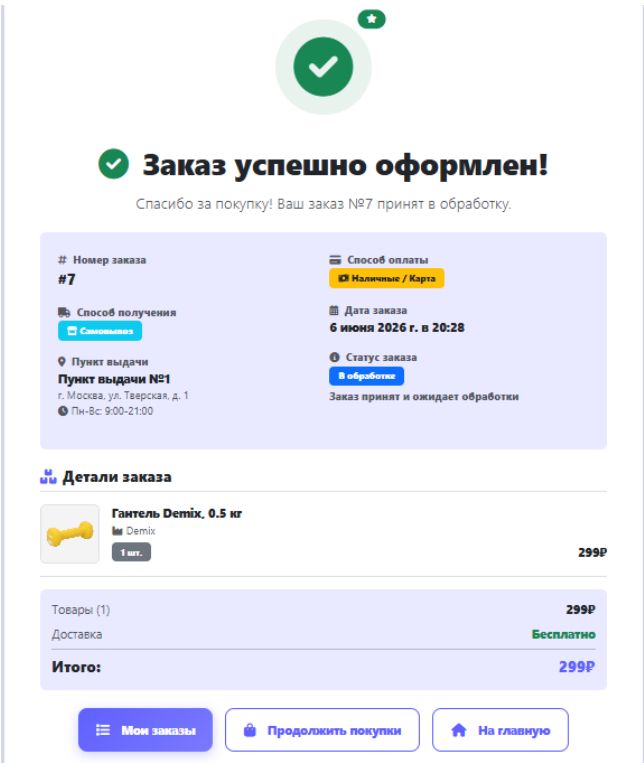


Рис. 38 Страница оформленного заказа

2.10. Личный кабинет

Личный кабинет предоставляет пользователю возможность управлять своим профилем, просматривать историю заказов и менять логин и аватар. Доступ к личному кабинету защищён middleware `isAuthenticated`.

Маршрут `/acc_page` отображает страницу профиля. При загрузке страницы из базы данных извлекаются данные пользователя (имя, email, аватар) и список его заказов. Для получения заказов используется функция `getOrdersByCustomer` из `database.js`, которая возвращает все заказы пользователя с их статусами.

```
<div class="container def-bg-settings py-4">
  <div class="row">
    <!-- Основной контент -->
    <div class="col-12">
      <!-- Заголовок и статус -->
      <div class="d-flex justify-content-between align-items-center mb-4">...
    </div>

    <!-- Основная карточка профиля -->
    <div class="card border-0 shadow-lg rounded-4 overflow-hidden mb-4 no-hover">
      <!-- Верхняя панель с градиентом -->
      <div class="profile-header-bg" style="height: 120px; background: linear-gradient(135deg, var(--primary-color), ...);">
      </div>

      <!-- Контент профиля -->
      <div class="card-body p-4" style="margin-top: -60px;">...
    </div>
  </div>
</div>
```

Рис. 39 EJS представление страницы аккаунта

Так же если пользователь авторизован как администратор, то в профиле появляется соответствующий бейдж.

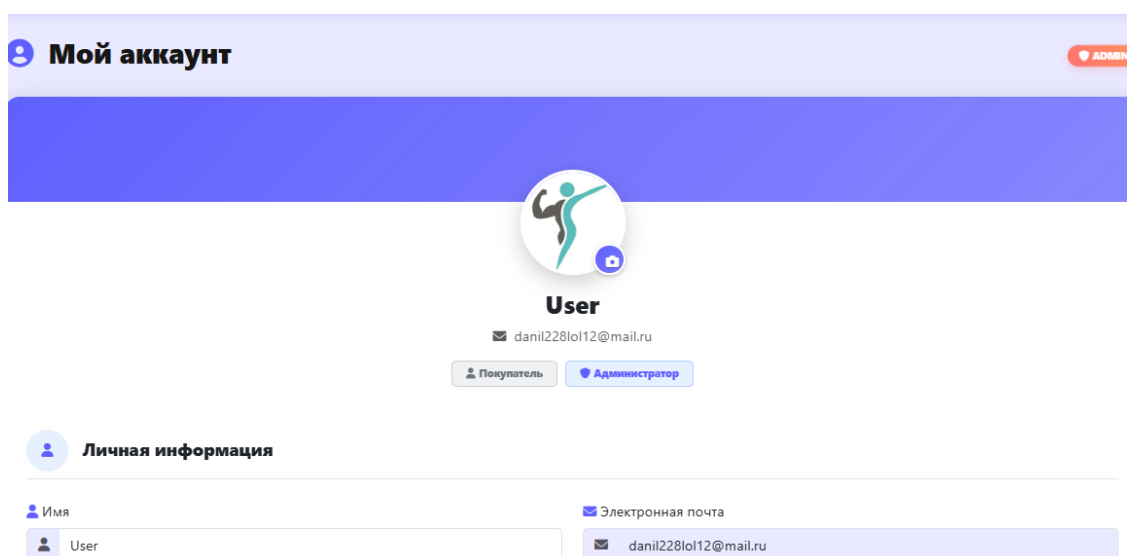


Рис. 40 Страница профиля администратора

У каждого пользователя из страницы профиля есть доступ к корзине, избранному истории заказов, благодаря блоку «быстрые действия»

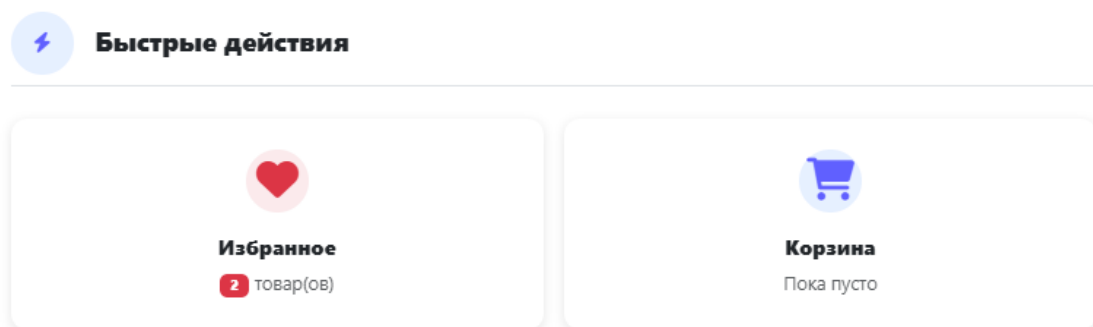


Рис. 41 Блок «быстрые действия»

2.11. Главная страница

Главная страница (маршрут /) является витриной магазина и должна привлекать внимание пользователя к ключевым товарам и категориям. При разработке были реализованы следующие блоки:

- Карусель - сочные изображения для привлечения внимания пользователя
- Популярные товары - отображаются товары с наивысшим рейтингом. Данные получаются через функцию `getPopularProducts(8)` из `database.js`, которая выбирает товары, сортирует их по `productRating DESC` и ограничивает количество до 8.
- Товары со скидкой - отображаются товары, у которых флаг `is_on_sale = 1` и текущая дата попадает в интервал `discount_start_date – discount_end_date`.
Функция `getDiscountedProducts(6)` возвращает до 6 таких товаров, отсортированных по размеру скидки (от большего к меньшему).
- Категории товаров - выводятся первые три, а остальные скрываются за ссылкой «Другие категории». Каждая категория имеет название и иконку (`categorieThumbnail`).

```
<!-- Основной контент -->
<div class="container def-bg-settings">
  <!-- Карусель / Слайдер -->
  <div class="hero-carousel mb-5"> ...
  </div>

  <!-- Раздел "Категории" -->
  <section class="categories-section mb-5"> ...
  </section>

  <!-- Раздел "Товары со скидкой" -->
  <% if (discountedProducts && discountedProducts.length > 0) { %>
  <section class="discounted-catalog-section mb-5"> ...
  </section>
  <% } %>

  <!-- Раздел "Товары с лучшими отзывами" -->
  <section class="reviews-section mb-5"> ...
  </section>
</div>
```

Рис. 42 Структура главной страницы

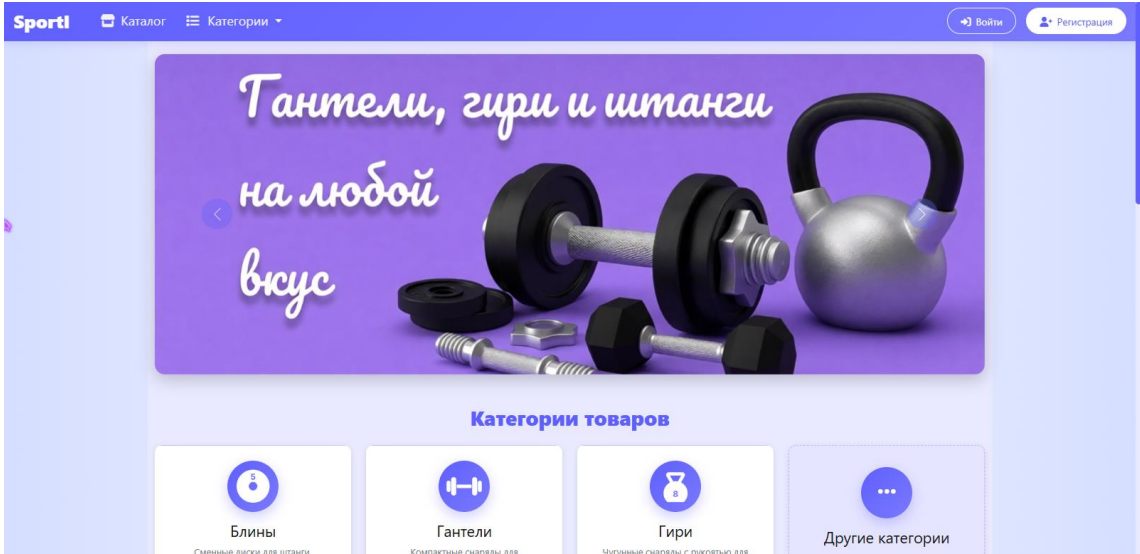


Рис. 42 Карусель на главной странице

ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломной работы было разработано полнофункциональное веб-приложение для продажи спортивного инвентаря «SportI». Результатом работы является готовый к эксплуатации интернет-магазин, реализующий все ключевые процессы электронной коммерции: от просмотра каталога до оформления заказа.

В процессе разработки были решены следующие задачи:

1. Проведён анализ предметной области – изучены современное состояние рынка спортивных товаров, тенденции и потребности целевой аудитории. Выявлены сильные и слабые стороны существующих решений (Спортмастер, Триал-спорт, Кант), что позволило сформулировать требования к собственному приложению.
2. Разработана архитектура приложения – выбрана монолитная клиент-серверная архитектура с использованием Node.js, Express.js и MySQL. Обоснован выбор технологического стека, описаны принципы организации кода (MVC, middleware, сессии).
3. Спроектирована и реализована реляционная база данных – создана нормализованная схема из 13 таблиц (пользователи, товары, категории, корзина, избранное, отзывы, заказы и др.), обеспечена ссылочная целостность внешними ключами и каскадным удалением.
4. Разработан пользовательский интерфейс – с использованием Bootstrap 5 и кастомных стилей созданы адаптивные страницы: главная (с каруселью, блоками популярных товаров и товаров со скидкой), каталог с фильтрацией и пагинацией, страница товара (с отзывами и характеристиками), личный кабинет, корзина, избранное, оформление заказа, административная панель.

5. Реализована бизнес-логика – система регистрации/авторизации (хеширование паролей bcrypt, сессии), управление корзиной и избранным (AJAX-запросы без перезагрузки), динамическая фильтрация и поиск с автодополнением, система отзывов и рейтингов, управление товарами и категориями для администратора (CRUD-операции, загрузка изображений через formidable), оформление заказа с проверкой остатков и транзакционной записью в БД.
6. Обеспечена безопасность – защита от SQL-инъекций (параметризованные запросы mysql2), от XSS (экранирование в EJS), от несанкционированного доступа (middleware isAuthenticated и checkAdmin), хеширование паролей, флаг httpOnly для кук сессии.

В результате создано веб-приложение, которое:

- обладает интуитивно понятным интерфейсом и адаптивной вёрсткой;
- поддерживает полный цикл покупки товара (выбор, корзина, оформление заказа);
- предоставляет администратору удобные инструменты для управления ассортиментом и категориями;
- масштабируемо и может быть дополнено новыми функциями (онлайн-оплата, интеграция с платежными системами, модуль рассылок, панель аналитики).

Разработанный продукт может быть использован как основа для реального интернет-магазина спортивного инвентаря, а также как учебный пример для изучения современных технологий веб-разработки.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. OWASP Top 10 – 2021 [Электронный ресурс]. – Режим доступа: <https://owasp.org/Top10/ru/>, свободный. – (дата обращения: 15.10.2025).
2. Браун, Э. Node.js в действии / Э. Браун. – 2-е изд. – М. : ДМК Пресс, 2022. – 448 с.
3. Верстак, В. А. Веб-разработка на Node.js и Express / В. А. Верстак. – СПб. : Питер, 2023. – 384 с.
4. Голдберг, К. Разработка веб-приложений на JavaScript / К. Голдберг. – М. : Эксмо, 2022. – 512 с.
5. Дакетт, Дж. HTML и CSS. Разработка и дизайн веб-сайтов / Дж. Дакетт. – М. : Эксмо, 2023. – 512 с.
6. Документация Bootstrap 5 [Электронный ресурс]. – Режим доступа: <https://getbootstrap.com/docs/5.3/>, свободный. – (дата обращения: 15.10.2025).
7. Документация EJS [Электронный ресурс]. – Режим доступа: <https://ejs.co/>, свободный. – (дата обращения: 15.05.2025).
8. Документация Express.js [Электронный ресурс]. – Режим доступа: <https://expressjs.com/ru/>, свободный. – (дата обращения: 15.05.2025).
9. Документация MySQL [Электронный ресурс]. – Режим доступа: <https://dev.mysql.com/doc/refman/8.0/en/>, свободный. – (дата обращения: 15.10.2025).
10. Документация Node.js [Электронный ресурс]. – Режим доступа: <https://nodejs.org/ru/docs/>, свободный. – (дата обращения: 15.10.2025).
11. Иванов, П. В. Современные методы разработки веб-приложений / П. В. Иванов. – М. : Лаборатория знаний, 2024. – 336 с.

12. Кумар, А. Создание решений электронной коммерции на Node.js / А. Кумар. – М. : ДМК Пресс, 2023. – 352 с.
13. Лаптев, В. В. Веб-дизайн и юзабилити. Современные стандарты / В. В. Лаптев. – СПб. : БХВ-Петербург, 2023. – 288 с.
14. Мартынов, П. И. Проектирование пользовательских интерфейсов: UX/UI для веба / П. И. Мартынов. – М. : ДМК Пресс, 2023. – 256 с.
15. Никсон, Р. Создание динамических веб-сайтов с помощью PHP, MySQL, JavaScript, CSS и HTML5 / Р. Никсон. – 6-е изд. – СПб. : Питер, 2023. – 704 с.
16. Прохоренок, Н. А. HTML, JavaScript, PHP и MySQL. Джентельменский набор Web-мастера / Н. А. Прохоренок. – 5-е изд. – СПб. : БХВ-Петербург, 2023. – 768 с.
17. Сидоров, В. В. Разработка серверных приложений на Node.js / В. В. Сидоров. – М. : ДМК Пресс, 2023. – 400 с.
18. Флэнаган, Д. JavaScript. Подробное руководство / Д. Флэнаган. – 7-е изд. – СПб. : Питер, 2022. – 720 с.
19. Фрейман, Э. Head First. Изучаем HTML и CSS / Э. Фрейман, Э. Робсон. – 3-е изд. – СПб. : Питер, 2022. – 720 с.
20. Холл, М. Секреты Node.js. Масштабируемые веб-приложения / М. Холл. – М. : Эксмо, 2024. – 368 с.